

Scenario 2: The bridge is floating

Slug: i90-span-clear - **Repo:** github.com/joelcapgemini/titan-watch-lab - **Your role:** incident commander, full authority, next page

The kaiju, in thirty seconds

- **Terrakon.** Tectonic Behemoth, 120 m, 67,000 t, 22.5 km/h, faster near shore. Walks the lake bed, surfaced. Declared target Mercer Island. The I-90 span is in the way. Does not stop. Cannot be stopped.
- **26 minutes** to the "anchor line". Titan Watch's phrase and figure, straight-line, refreshed every 30 seconds. Usually runs early. Nobody has confirmed what that is.
- **The span is lost twice.** Unsafe for traffic first, then broken. Its anchors feel the footfalls before it arrives.
- **Who sees it.** Satellite sees where it is, every 30 seconds. Observers see what it does, not where. Engineers see strain, not cause. Nobody sees all of it. That gap is the scenario.

4,000 vehicles and maybe two trains are on the span. Clear it and strand Mercer Island, or keep it open and gamble 4,000 vehicles on a cable.

Outcome, same for every scenario

Save as much human life as possible, then prevent as much public property damage as possible, in that order. A software tool is expected to come out of this.

You are graded on, in this order: people on the span when it goes; people stranded on Mercer Island without warning; damage to the bridge and its anchors; and whether every decision you made was a decision, recorded, with a name on it.

The coach interviews you against this, narrows it to this clock and this span, and restates it after synthesis. The original stays in the scope file beside the restatement.

Who reports to you

Nine specialists have already been interviewed. Their interviews are in `.copilot-tracking/dt/i90-span-clear/`. Do not read them. The coach has, and will quiz you on what they said.

Reports to you	What they own	What they keep saying
WSDOT traffic management centre	Signs, signals, gates, cameras	Four minutes before the road changes; flow, not count
WSDOT structural engineer	The call that the span is unsafe	Two thresholds; nobody computes the first
State Patrol ground commander	Ramps	Cannot reach anyone already on the span
Mercer Island emergency manager	25,000 people on the far side	Learns from the news; east span unknown
Sound Transit rail control	Trains, 400 people each	Cannot divert; hold before the closure
Titan Watch analyst and observer	The clock	26 is the island, not the bridge; runs early; stale is silent
A driver mid-span	One car, four people	Green signal means the road is open
Tow and recovery	Vehicles that will not move	Four stalls halve the drain; on no list

Already done, do not redo

Scope, stakeholder map, assumptions log, world rules, nine interviews, security scoping answers. All under `.copilot-tracking/dt/i90-span-clear/`. The coach reads them.

Out of scope for everyone: engaging Terrakon, island shelter, rebuilding the span.

Your card: Incident commander, I-90 span

Scenario 2, I-90 span. Kaiju facts are on page 1 of this pack. Human facts are yours: invent and label.

Outcome: save as much human life as possible, then prevent as much public property damage as possible, in that order. The coach narrows it; that is fine.

Slug: i90-span-clear-commander

Who you are

You are in charge. WSDOT, State Patrol, Sound Transit, Mercer Island and the Titan Watch liaison all answer to you for the next 26 minutes. You can close the span, hold any train, block any ramp, send any truck anywhere, and tell the island what you choose. Nobody will argue. The only things you do not control are Terrakon and the clock.

Who you can call, and for what

- TMC: signs, signals, gates, cameras, drain time.
- Engineer: when the span is unsafe, when it fails.
- State Patrol: ramps blocked, and when.
- Rail control: trains on the span, last release time.
- Mercer Island: what it knows, east span open or not.
- Tow: vehicles that will not move.
- Titan Watch liaison: the clock, what it counts to, how stale it is.

Each sees a piece. You see what they tell you, when they tell you. No screen of your own.

Your scope

In scope, you decide and order:

- Close the span, yes or no, and when.
- Which trains hold, which ramps shut, which way drivers go.
- When to tell Mercer Island what, and when to stop clearing and start choosing.

Out of scope:

- How a specialist carries out your order. They know their job.
- Where Terrakon is or goes. Titan Watch reports it; nobody controls it.

Anchors, fixed

- Every order you give takes minutes to reach the people who act on it, and every report reaches you late.
- No one has ever cleared this span against a clock, or computed when it becomes unsafe.

Three phrases

"I do not know." - "Assume and label it." - "Ask the specialist, then tell me."

Next

Turn the page. The run sheet has every command and prompt for this scenario, in order.

How to proceed: I-90 bridge

Order of work, with every prompt filled in for this scenario. Full detail is in the shared instructions.

Lab 1: Discovery

1 /clear, then /hve-core:dt-coach. Paste:

Resume project i90-span-clear. Scope is done under .copilot-tracking/dt/i90-span-clear/; read it, including every interview there, do not redo it. I play the incident commander with full authority: the specialists in those interviews report to me and carry out my orders. I see only what they report. Interview me as that role, five questions, using what the specialists said. Then write interview-commander-roleplay.md under slug i90-span-clear-commander. Label everything role-play. Expect a software tool. Do not offer the canonical deck.

1 Answer five questions as the commander. Read your interview file once.

2 Same session, paste:

Project i90-span-clear. Read the shared scope under .copilot-tracking/dt/i90-span-clear/, every interview there, and my interview under .copilot-tracking/dt/i90-span-clear-commander/. Restate the outcome in light of the interviews, keeping the original beside it. Summarise the problem as if hearing it for the first time. List what you still need before a PRD. Then draft the PRD with non-functional requirements, close gaps with labelled assumptions, and state what it rests on. Then a technology-neutral diagram as Mermaid. Then an opinionated Azure overlay with the minimum services, rejected alternatives recorded, and a latency budget. Write everything under .copilot-tracking/dt/i90-span-clear/. Run without stopping. Label all interviews role-play.

1 PRD, diagram and overlay land under dt/i90-span-clear/.

Lab 2: Security and Responsible AI

Write three predicted security findings on paper first. Two terminals.

- 1 Terminal 1: `/clear`, then `/hve-core:security-planner`. Paste:

Use the security planner to plan the production security for this solution. PRD and architecture are under `.copilot-tracking/dt/i90-span-clear/`. Scoping answers are in `security-scoping-answers.md` in the same folder; treat every row as answered and labelled, do not ask them again. Threat-model the boundaries, map controls to OWASP, NIST and CIS, produce a backlog. Ask me only about severe or blocking issues. Run all phases without stopping. Prioritise by severity. Do not commit or push.

- 1 Terminal 2: `/clear`, then `/hve-core:rai-planner-agent`. Paste:

Run an RAI assessment on the single-save decision: the disaster-recovery AI in this app's Last Stand scene that recommends which one of two cities to save when only one can be saved. Today it outputs a single verdict with no confidence, no explanation and no audit trail, and a commander approves it under time pressure. Assess it against NIST AI RMF and treat the autonomous life-safety decision as the central risk. Use project name `single-save-decision`. Imagine the scoping answers, label them, run to completion unless blocking.

- 1 When the security planner finishes: "Plan summary by severity: counts per level, Critical items one line each, recommended next planner." Compare with your paper. Read the RAI go/no-go.

Lab 3: Research, Plan, Implement

- 1 Open your PRD under `dt/i90-span-clear/`. Pick one functional requirement the app could carry. One screen, one rule, one feed.
- 2 `/clear`, then `/hve-core:rpi-agent`. Ask: "What do you need from me to create an RPI plan?" Then paste, filled in:

Goal: implement <requirement, one sentence> from the PRD at `.copilot-tracking/dt/i90-span-clear/<prd file>`.
Scope: <one component or module>. Nothing else.
Acceptance criteria: <two or three testable lines>.
Constraints: no backend, no tracker, no cloud tenant; mock or defer and log the gap. No tracking paths in code or commits.
Validation: the existing test, lint and build scripts.
Context: the PRD and the Azure overlay in the same folder.
Mode: research, stop for my review; plan, stop for my approval.

- 1 Research stops: read it, fix the file if wrong, then `/hve-core:rpi-plan`.
- 2 Plan stops: amend one item in the plan file, tell the agent "plan amended in the file".
- 3 Homework: `/hve-core:rpi-implement`, then `/hve-core:rpi-review`.